PHASE 1

Functional Requirements (System Operations)

The centralized PostgreSQL database must natively support the following core workflows:

1. Student Registration and Profile Management: Store detailed personal, contact, and enrollment metadata for every student.
2. Lecturer Registration and Departmental Assignment: Manage lecturer credentials, contact info, and their administrative assignment to primary academic departments.
3. Course Creation and Departmental Allocation: Support the design of the university curriculum by defining courses, assigning credit weightings, and allocating them to their respective host departments.
4. Student Enrollment in Courses: Record and track the registration of students in specific course sections per semester.
5. Scheduling and Allocation of Academic Spaces: Coordinate reservation times, recurring class schedules, and specific room assignments.
6. Attendance/Participation Tracking: Provide a unified mechanism to record attendance logs for academic course sessions.
7. Research Project Supervision & Milestones: Support tracking of final-year undergraduate and graduate research projects, including the dynamic JSONB metadata mapping of milestones.
8. Academic Event Creation and Participation: Support the configuration of conferences, symposiums, and public lectures, tracking registration from both students and lecturers.
9. Resource and Space Tracking: Audit resource types, physical location, and safe capacity thresholds across all departments.

Non-Functional Requirements (System Qualities)

1. Data Integrity across Relationships: Enforce strict transactional integrity. Orphaned child records must be prevented using appropriate cascading behaviors (ON DELETE RESTRICT / ON DELETE CASCADE).
2. Efficient Query Execution**:** The system must utilize structured indices (B-trees and GIN indices) to handle high-frequency reads over tables with hundreds of thousands of records.

3. Concurrency Control: Leverage PostgreSQL's Multi-Version Concurrency Control (MVCC) to allow concurrent student enrollments without race conditions, write starvation, or phantom reads.
4. Scalability: Structure schema entities to allow horizontal scaling and table partitioning (e.g., archiving historic attendance and enrollment records) as university registration grows.
5. Security and Access Control: Maintain structural separation of confidential data (e.g., student grades, employee ranks, and budgets) through schemas and views with defined privileges.
6. Referential and Domain Consistency: Utilize check constraints, custom domains, and relational triggers to ensure academic invariants (e.g., event capacities are never exceeded and grades remain within $[0, 100]$ limits).

System Assumptions

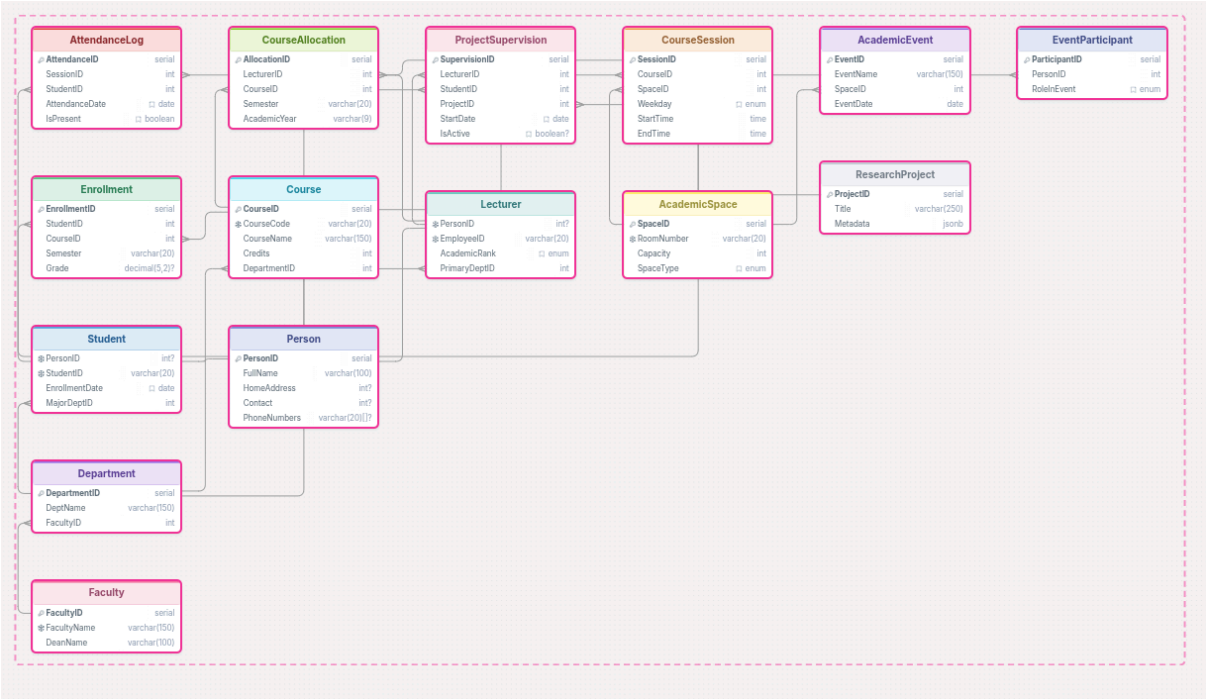
To establish boundaries for database schema modeling, the following rules and operational boundaries are defined:

- Academic Structure: A faculty can house multiple departments. A department belongs to exactly one faculty.
    - A student belongs to exactly one major department at any given time.
    - A lecturer belongs to exactly one primary department for payroll/administrative purposes, but can teach courses hosted by other departments.
- Courses & Class Allocations:
    - A course can have multiple active sections.
    - Classrooms and laboratories represent physical "academic spaces." No two sessions can be scheduled in the same academic space at the same time.
- Research & Supervision:
    - A student's final-year research project must have exactly one primary lecturer supervisor.
    - A lecturer can supervise multiple research projects concurrently.
- Event Participation:
    - An academic event is open to both students and lecturers. Both groups register under the shared role of EventParticipant.

Phase 2

Conceptual Design (ER Model)

Entity Identification & Attribute Specification



Entity Additions Justification

- Person (Base Entity): Introduced to model the inheritance pattern. Both students and lecturers share foundational physical identity metadata but are split into specialized behavioral classes.
- Project Supervision (Aggregate Entity): Modeled explicitly because final-year supervisions represent a dynamic administrative contract linking a student, a supervisor, and a research project. Treating this as a high-level entity permits future expansion (such as tying milestone progress reports or external funding bodies directly to the supervision instance).
- EventParticipant (Category/Union Entity): Created to prevent duplicate structural relationships. Since both students and lecturers can register for, present at, or attend events, registering them under a single polymorphic participant entity normalizes event scheduling.

Conceptual Relationship Specifications

To map the physical interactions of our system, we define the following 12 relationships:

1. Faculty to Department

- Verb Phrase: *Faculty houses Departments*
- Cardinality Ratio & Constraints: 1:M (One Faculty, Many Departments). Mandatory on the Department side (a department must belong to a faculty).

2. Department to Student

- Verb Phrase: *Student belongs to the Department*
- Cardinality Ratio & Constraints: 1:M (One Department, Many Students). Mandatory on the Student side.

3. Department to Lecturer

- Verb Phrase: *The lecturer is assigned to the Department*
- Cardinality Ratio & Constraints: 1:M (One Department, Many Lecturers). Mandatory on the lecturer's side.

4. Department to Course

- Verb Phrase: The *Department offers the course.*
- Cardinality Ratio & Constraints: 1:M (One Department, Many Courses). Mandatory on the course side.

5. Student to Course (via Enrollment)

- Verb Phrase: *Student enrolls in Course*
- Cardinality Ratio & Constraints: M:N (Many-to-Many). Modeled via the enrollment bridge entity, where each enrollment record maps one student to one course per semester.

6. Lecturer to Course (Allocation)

- Verb Phrase: *Lecturer teaches Course*
- Cardinality Ratio & Constraints: M:N (Many lecturers can teach a course, and a lecturer can teach multiple courses).

7. Course to Academic Space (Scheduling)

- Verb Phrase: *The course is scheduled in the academic space.*
- Cardinality Ratio & Constraints: M: N (Many courses use various spaces over a week). Modeled using a schedule bridging table to prevent double-booking.

8. Project Supervision Aggregation

- Verb Phrase: *Lecturer supervises Student Research Project*
- Cardinality Ratio & Constraints: A ternary relationship aggregated into a single entity.
  A student works on one project under one lecturer; a lecturer can oversee many such project supervisions.

9. Academic Space to Academic Event

- Verb Phrase: *The event is scheduled in the academic space.*
- Cardinality Ratio & Constraints: M:1 (Many events can be hosted in an academic space over time, but an event takes place in one physical space at a given time).

10. Person to EventParticipant (Category Union mapping)

- Verb Phrase: *Person registers as EventParticipant*
- Cardinality Ratio & Constraints: **A** 1:1 mapping from the specific person (whether student or lecturer) to the polymorphic EventParticipant record.

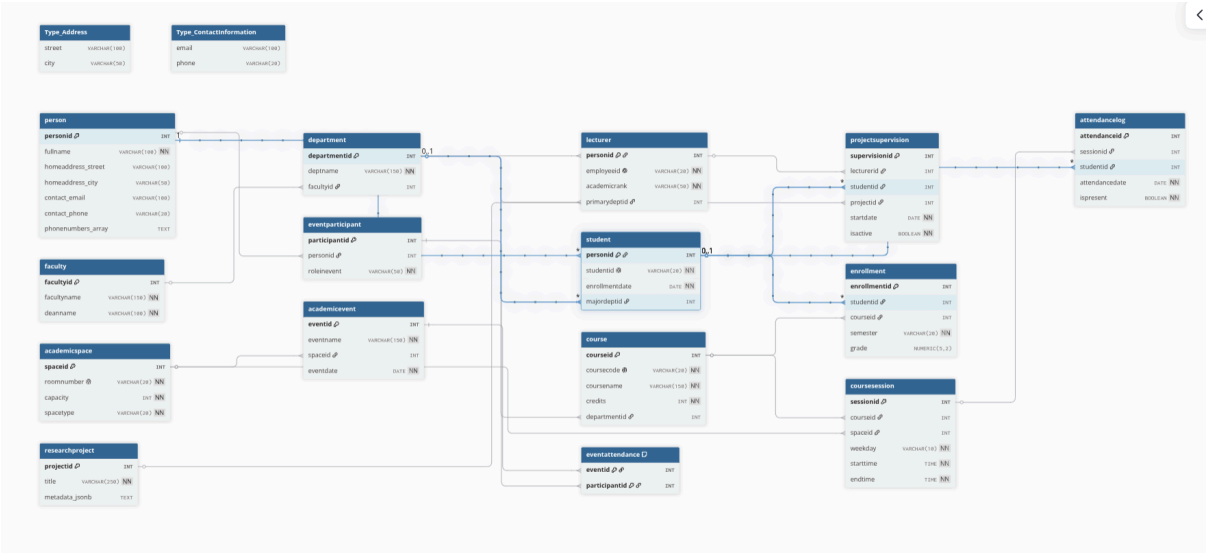
11. EventParticipant to Academic Event (via EventAttendance)

- Verb Phrase: *Participant attends Academic Event*
- Cardinality Ratio & Constraints: M: N (Many participants attend many events). Modeled through EventAttendance.

12. Student attends Course Session (Attendance tracking)

- Verb Phrase: *Student attends Course Session*
- Cardinality Ratio & Constraints: M: N relationship tracking active weekly presence profiles.

Phase 3: Enhanced Entity-Relationship (EER) Modeling



Specialization

We define a specialization hierarchy centered around the core actors within ULK:

- Superclass: Person
    - Inherited Attributes: personid (Primary Key), fullname, homeaddress (Composite Object Type: address), contact (Composite Object Type: contact information), and phonenumbers (Array Attribute representing secondary numbers).
- Subclasses: Student and Lecturer
    - Role-Specific Attributes (Student): studentid (unique structural card identifier), enrollmentdate (defaulting to current database register date), and majordeptid (foreign key connecting the student to their major academic department).
    - Role-Specific Attributes (Lecturer): employeeid (unique employee payload code), academicrank (validated through a domain constraint set: Tutorial Assistant, Assistant Lecturer, Lecturer, Senior Lecturer, Associate Professor, or Professor), and primarydeptid (foreign key connecting the supervisor to their primary parent department).

Constraints and Justification:

- Disjointness Constraint: Disjoint (D). At Kigali Independent University, an individual record within this system is strictly classified as either a student or a lecturer. Although a lecturer might take courses or a student might serve as an assistant, they utilize fundamentally separated ID registration protocols, payroll streams, and functional roles. Thus, to ensure system-wide constraints and operational security, they are modeled as disjoint.
- Completeness Constraint: Total (T). Every personal record managed within the central academic registry *must* belong to one of these two key specialized branches.

Generalization (Logically Modeled via Room Types)

Rather than splitting physical space across fragile structural tables that increase join overhead, facilities are generalized into a centralized table.

- Entity: AcademicSpace
    - Common Attributes: spaceid (Primary Key), roomnumber (unique building location index), capacity (positive seating limit check), and spacetype (a strict check domain indicating whether the room is a standard classroom or a specialized laboratory).

Aggregation

In academic systems, relationships can link to other relationships. We model project supervision as an aggregation:

- Core Relationship: A student undertakes a research project.
- Aggregation: This entire relationship [Student - Undertakes - Research Project] is aggregated into a higher-level conceptual entity.
- Aggregated Relationship: This composite object is connected to a lecturer via a "Supervises" relationship, physically realized in the table project supervision.

Category (Union Type)

We define a union type category to manage university-wide events:

- Category Class: EventParticipant
- Union Members: Student $\cup$ Lecturer

- Attributes: participantid (Primary Key), personid (reference tracking back to the base person table), roleinevent (strict checklist validation: Keynote Speaker, Presenter, Evaluation Panelist, Attendee).

Phase 4 and 5: Are implemented in the sql file

Phase 6:

Query 1: Selection and Projection (Basic Academic Filtering)

```
SELECT studentid, fullname, enrollmentdate
FROM public.student
WHERE enrollmentdate < '2023-09-01'
ORDER BY enrollmentdate ASC;
```

Query 2: Custom Composite Type Querying

```
SELECT personid,
    fullname,
    (homeaddress).street AS street_name,
    (contact).email AS primary_email
FROM public.student
WHERE (homeaddress).city = 'Kigali';
```

Query 3: Multi-Table Joins (Structural Departmental Hierarchy)

```
SELECT f.facultyname AS faculty_name,
    f.deanname AS faculty_dean,
    d.deptname AS department_name
FROM public.department d
INNER JOIN public.faculty f ON d.facultyid = f.facultyid
ORDER BY f.facultyname, d.deptname;
```

Query 4: Left Join with Null Verification (Unallocated Curriculum Audit)

```
SELECT c.coursecode, c.coursename, c.credits
FROM public.course c
LEFT JOIN public.courseallocation ca ON c.courseid = ca.courseid
WHERE ca.allocationid IS NULL;
```

Query 5: Multiple Aggregations & Grouping (Departmental Performance)

```
SELECT d.deptname AS department_name,
    COUNT(e.enrollmentid) AS total_enrolled_exams,
    ROUND(AVG(e.grade), 2) AS average_grade,
    MIN(e.grade) AS minimum_grade,
    MAX(e.grade) AS maximum_grade
FROM public.department d
INNER JOIN public.student s ON d.departmentid = s.majordeptid
INNER JOIN public.enrollment e ON s.personid = e.studentid
WHERE e.grade IS NOT NULL
GROUP BY d.departmentid, d.deptname
ORDER BY average_grade DESC;
```

Query 6: Having Clause (Academic Space Resource Capacity Audit)

```
SELECT spacetype,
    COUNT(spaceid) AS total_rooms,
    ROUND(AVG(capacity), 1) AS average_capacity
FROM public.academicspace
GROUP BY spacetype
HAVING AVG(capacity) > 45;
```

Query 7: Non-Correlated Subquery (Outstanding Cohort Honors)

```
SELECT s.studentid, s.fullname, c.coursecode, c.coursename, e.grade, e.semester
FROM public.student s
INNER JOIN public.enrollment e ON s.personid = e.studentid
INNER JOIN public.course c ON e.courseid = c.courseid
WHERE e.grade = (
   SELECT MAX(grade)
   FROM public.enrollment
);
```

Query 8: Correlated Subquery (Peer Performance Benchmarking)

```
SELECT s.fullname AS student_name,
      c.coursename AS course_name,
      e1.grade AS student_grade,
      ROUND((SELECT AVG(e2.grade) FROM public.enrollment e2 WHERE
e2.courseid = e1.courseid), 2) AS course_average
FROM public.enrollment e1
INNER JOIN public.student s ON e1.studentid = s.personid
INNER JOIN public.course c ON e1.courseid = c.courseid
WHERE e1.grade > (
   SELECT AVG(e2.grade)
   FROM public.enrollment e2
   WHERE e2.courseid = e1.courseid
)
ORDER BY c.coursename, e1.grade DESC;
```

Query 9: Advanced JSONB Document Traversals (Research Tag Lookup)

```
SELECT projectid,
    title,
    metadata->>'status' AS project_status,
    metadata->'funding'->>'source' AS funding_body,
    (metadata->'funding'->>'amount_USD')::numeric AS funding_amount_usd
FROM public.researchproject
WHERE metadata->'tags' @> '"AI"'::jsonb
  OR metadata->'tags' @> '"Computer Vision"'::jsonb;
```

Query 10: Array Attribute Traversal (Lecturers with Dual Numbers)

```
SELECT personid,
    fullname,
    (contact).email AS primary_email,
    unnest(phonenumbers) AS phone_number_entry
FROM public.lecturer
WHERE array_length(phonenumbers, 1) > 1;
```

Query 11: Set Operation (Union) across Inherited Tables

```
SELECT personid, fullname, (contact).email AS email, 'Student' AS registry_role
FROM public.student
UNION ALL
SELECT personid, fullname, (contact).email AS email, 'Lecturer' AS registry_role
ORDER BY fullname ASC;
```

Query 12: Aggregate with Window Functions (Course Performance Ranking)

```
SELECT c.coursecode,
```

```
    c.coursename,
    s.fullname AS student_name,
    e.grade,
    DENSE_RANK() OVER (
        PARTITION BY e.courseid
        ORDER BY e.grade DESC NULLS LAST
    ) AS class_rank
FROM public.enrollment e
INNER JOIN public.student s ON e.studentid = s.personid
INNER JOIN public.course c ON e.courseid = c.courseid
WHERE e.grade IS NOT NULL;
```

Query 13: Category (Union Type) Polymorphic Aggregation

```
SELECT ae.eventname,
    ae.eventdate,
    SUM(CASE WHEN ep.roleinevent = 'Keynote Speaker' THEN 1 ELSE 0 END)
AS keynote_speakers,
    SUM(CASE WHEN ep.roleinevent = 'Presenter' THEN 1 ELSE 0 END) AS
presenters,
    SUM(CASE WHEN ep.roleinevent = 'Attendee' THEN 1 ELSE 0 END) AS
general_attendees,
    COUNT(ea.participantid) AS total_participants
FROM public.academicevent ae
LEFT JOIN public.eventattendance ea ON ae.eventid = ea.eventid
LEFT JOIN public.eventparticipant ep ON ea.participantid = ep.participantid
GROUP BY ae.eventid, ae.eventname, ae.eventdate
```

```
ORDER BY ae.eventdate ASC;
```

Query 14: Complex Conditional Aggregation (Student Engagement Ratios)

```
SELECT s.fullname AS student_name,
    c.coursename AS course_name,
    e.grade AS final_grade,
    COUNT(al.attendanceid) AS sessions_tracked,
    SUM(CASE WHEN al.ispresent = TRUE THEN 1 ELSE 0 END) AS
sessions_present,
    ROUND(
       (SUM(CASE WHEN al.ispresent = TRUE THEN 1 ELSE 0 END)::numeric /
        NULLIF(COUNT(al.attendanceid), 0)::numeric) * 100, 2
    ) AS attendance_rate_pct
FROM public.student s
INNER JOIN public.enrollment e ON s.personid = e.studentid
INNER JOIN public.course c ON e.courseid = c.courseid
INNER JOIN public.coursesession cs ON c.courseid = cs.courseid
LEFT JOIN public.attendancelog al ON cs.sessionid = al.sessionid AND s.personid =
al.studentid
GROUP BY s.personid, s.fullname, c.courseid, c.coursename, e.grade
ORDER BY student_name, course_name;
```

Query 15: Cross-Table Scheduling Overlap Check

```
SELECT l.fullname AS lecturer_name,
    cs1.weekday AS conflict_day,
    c1.coursecode AS first_course,
    cs1.starttime AS first_start,
```

```
    cs1.endtime AS first_end,
    c2.coursecode AS second_course,
    cs2.starttime AS second_start,
    cs2.endtime AS second_end,
    asp.roomnumber AS physical_room
FROM public.coursesession cs1
INNER JOIN public.coursesession cs2 ON cs1.weekday = cs2.weekday
  AND cs1.sessionid < cs2.sessionid -- Prevents checking a session against itself or listing duplicates
INNER JOIN public.course c1 ON cs1.courseid = c1.courseid
INNER JOIN public.course c2 ON cs2.courseid = c2.courseid
INNER JOIN public.courseallocation ca1 ON c1.courseid = ca1.courseid
INNER JOIN public.courseallocation ca2 ON c2.courseid = ca2.courseid AND ca1.lecturerid = ca2.lecturerid
INNER JOIN public.lecturer l ON ca1.lecturerid = l.personid
INNER JOIN public.academicspace asp ON cs1.spaceid = asp.spaceid
WHERE (cs1.starttime, cs1.endtime) OVERLAPS (cs2.starttime, cs2.endtime);
```

Phase 7: Query Processing Analysis

Query A: Course Standing & Performance Grading (Query 12)

This query ranks student performance within each course based on final grades.

```
EXPLAIN ANALYZE
SELECT c.coursename, s.fullname, e.grade,
    RANK() OVER (PARTITION BY e.courseid ORDER BY e.grade DESC)
AS course_standing
```

```
FROM public.enrollment e
JOIN public.student s ON e.studentid = s.personid
JOIN public.course c ON e.courseid = c.courseid
WHERE e.grade IS NOT NULL;
```

Execution Plan Structure

```
WindowAgg  (cost=13.03..15.03 rows=101 width=58) (actual
time=0.158..0.198 rows=101.00 loops=1)
"  Window: w1 AS (PARTITION BY e.courseid ORDER BY e.grade ROWS
UNBOUNDED PRECEDING)"
"  Storage: Memory  Maximum Storage: 17kB"
"  Buffers: shared hit=11"
"  ->  Sort  (cost=13.01..13.26 rows=101 width=50) (actual time=0.150..0.155
rows=101.00 loops=1)"
"        Sort Key: e.courseid, e.grade DESC"
"        Sort Method: quicksort  Memory: 32kB"
"        Buffers: shared hit=11"
"        ->  Hash Join  (cost=6.92..9.65 rows=101 width=50) (actual
time=0.075..0.108 rows=101.00 loops=1)"
"              Hash Cond: (e.courseid = c.courseid)"
"              Buffers: shared hit=5"
"              ->  Hash Join  (cost=5.25..7.68 rows=101 width=26) (actual
time=0.048..0.069 rows=101.00 loops=1)"
"                    Hash Cond: (e.studentid = s.personid)"
"                    Buffers: shared hit=4"
"                    ->  Seq Scan on enrollment e  (cost=0.00..2.15 rows=101
width=13) (actual time=0.007..0.015 rows=101.00 loops=1)"
"                          Filter: (grade IS NOT NULL)"
"                          Rows Removed by Filter: 14"
"                          Buffers: shared hit=1"
"                    ->  Hash  (cost=4.00..4.00 rows=100 width=21) (actual
time=0.028..0.028 rows=100.00 loops=1)"
"                          Buckets: 1024  Batches: 1  Memory Usage: 14kB"
"                          Buffers: shared hit=3"
"                          ->  Seq Scan on student s  (cost=0.00..4.00 rows=100
width=21) (actual time=0.004..0.012 rows=100.00 loops=1)"
```

```
"                         Buffers: shared hit=3"
"            ->  Hash  (cost=1.30..1.30 rows=30 width=28) (actual
time=0.019..0.019 rows=30.00 loops=1)"
"                  Buckets: 1024  Batches: 1  Memory Usage: 10kB"
"                  Buffers: shared hit=1"
"                  ->  Seq Scan on course c  (cost=0.00..1.30 rows=30 width=28)
(actual time=0.008..0.010 rows=30.00 loops=1)"
"                        Buffers: shared hit=1"
Planning:
"  Buffers: shared hit=199 dirtied=3"
Planning Time: 0.708 ms
Execution Time: 0.308 ms
```

Explanation

This PostgreSQL query joins three tables, sorts the data, and applies a window function in 0.308 milliseconds entirely in memory.

Step-by-Step Execution

1. Scan & First Join: The database reads student and enrollment via sequential scans. It filters out null grades and joins them on student IDs.
2. Second Join: It scans the course table, builds a hash table, and joins it with the previous results on course IDs (yielding 101 rows).
3. Sort: It uses quicksort in RAM (32kB) to order the rows by courseid and grade DESC.
4. Window Function: The WindowAgg node computes the final window calculation, partitioning the sorted data by course.

Category Union and Event Registration

This query aggregates total student and lecturer presenter roles across all academic events.

```
EXPLAIN ANALYZE
SELECT ae.eventname,
     SUM(CASE WHEN ep.roleinevent = 'Presenter' AND s.personid IS NOT
NULL THEN 1 ELSE 0 END) AS student_presenters,
```

```
    SUM(CASE WHEN ep.roleinevent = 'Presenter' AND l.personid IS NOT
NULL THEN 1 ELSE 0 END) AS lecturer_presenters
FROM public.academicevent ae
JOIN public.eventattendance ea ON ae.eventid = ea.eventid
JOIN public.eventparticipant ep ON ea.participantid = ep.participantid
LEFT JOIN public.student s ON ep.personid = s.personid
LEFT JOIN public.lecturer l ON ep.personid = l.personid
GROUP BY ae.eventid, ae.eventname;
```

Execution Plan Structure

```
HashAggregate  (cost=140.48..142.68 rows=220 width=338) (actual
time=0.303..0.311 rows=7.00 loops=1)
"  Group Key: ae.eventid"
"  Batches: 1  Memory Usage: 32kB"
"  Buffers: shared hit=7"
"  ->  Hash Left Join  (cost=55.50..112.23 rows=2260 width=448) (actual
time=0.235..0.278 rows=26.00 loops=1)"
"        Hash Cond: (ep.personid = l.personid)"
"        Buffers: shared hit=7"
"        ->  Hash Left Join  (cost=42.12..92.80 rows=2260 width=448) (actual
time=0.199..0.233 rows=26.00 loops=1)"
"              Hash Cond: (ep.personid = s.personid)"
"              Buffers: shared hit=6"
"              ->  Hash Join  (cost=36.88..81.50 rows=2260 width=444) (actual
time=0.113..0.138 rows=26.00 loops=1)"
"                    Hash Cond: (ea.participantid = ep.participantid)"
"                    Buffers: shared hit=3"
"                    ->  Hash Join  (cost=14.95..53.59 rows=2260 width=326) (actual
time=0.071..0.086 rows=26.00 loops=1)"
"                          Hash Cond: (ea.eventid = ae.eventid)"
"                          Buffers: shared hit=2"
"                          ->  Seq Scan on eventattendance ea  (cost=0.00..32.60
rows=2260 width=8) (actual time=0.018..0.022 rows=26.00 loops=1)"
"                                Buffers: shared hit=1"
"                          ->  Hash  (cost=12.20..12.20 rows=220 width=322) (actual
time=0.025..0.026 rows=8.00 loops=1)"
"                                Buckets: 1024  Batches: 1  Memory Usage: 9kB"
```

```
"                         Buffers: shared hit=1"
"                         ->  Seq Scan on academicevent ae  (cost=0.00..12.20
rows=220 width=322) (actual time=0.010..0.012 rows=8.00 loops=1)"
"                               Buffers: shared hit=1"
"                   ->  Hash  (cost=15.30..15.30 rows=530 width=126) (actual
time=0.019..0.020 rows=20.00 loops=1)"
"                         Buckets: 1024  Batches: 1  Memory Usage: 10kB"
"                         Buffers: shared hit=1"
"                         ->  Seq Scan on eventparticipant ep  (cost=0.00..15.30
rows=530 width=126) (actual time=0.007..0.011 rows=20.00 loops=1)"
"                               Buffers: shared hit=1"
"               ->  Hash  (cost=4.00..4.00 rows=100 width=4) (actual
time=0.057..0.058 rows=100.00 loops=1)"
"                     Buckets: 1024  Batches: 1  Memory Usage: 12kB"
"                     Buffers: shared hit=3"
"                     ->  Seq Scan on student s  (cost=0.00..4.00 rows=100 width=4)
(actual time=0.011..0.033 rows=100.00 loops=1)"
"                           Buffers: shared hit=3"
"          ->  Hash  (cost=11.50..11.50 rows=150 width=4) (actual
time=0.016..0.016 rows=20.00 loops=1)"
"               Buckets: 1024  Batches: 1  Memory Usage: 9kB"
"               Buffers: shared hit=1"
"               ->  Seq Scan on lecturer l  (cost=0.00..11.50 rows=150 width=4)
(actual time=0.007..0.010 rows=20.00 loops=1)"
"                     Buffers: shared hit=1"
Planning:
"  Buffers: shared hit=239"
Planning Time: 1.740 ms
Execution Time: 0.517 ms
```

Explanation

This PostgreSQL query aggregates data across five tables, executing completely in memory in 0.517 milliseconds.

Step-by-Step Execution

1. Base Joins (Inner Hash Joins)

    - Event & Attendance: The database sequentially scans eventattendance (26 rows) and academicevent (8 rows), joining them on eventid.
    - Add Participants: It scans eventparticipant (20 rows) and inner joins it to the stream on participantid, resulting in 26 rows.
2. Optional Lookups (Left Hash Joins)
    - Student Check: It scans the student table (100 rows) into a hash table and performs a LEFT JOIN on personid.
    - Lecturer Check: It scans the lecturer table (20 rows) into a hash table and performs another LEFT JOIN on personid. The stream remains at 26 rows.
3. Final Aggregation
    - HashAggregate: It groups the remaining 26 rows by ae.eventid using a 32kB in-memory hash table, collapsing them into 7 final rows.

Performance & Issues

- The Good: Highly efficient caching. All 7 buffer hits occurred in RAM (shared hit), meaning zero slow disk I/O.
- The Bad: Massive statistics skew. The database predicted 2,260 rows (rows=2260), but actual loops only found 26 rows (rows=26.00). While harmless at this microscopic scale, this miscalculation will cause severe performance degradation (like unnecessary disk spilling) if your database grows.

Attendance Ratios and Conditional Grading

This query computes student session attendance ratios and maps them against course grades.

```
EXPLAIN ANALYZE
SELECT s.fullname, c.coursename, e.grade,
    COUNT(CASE WHEN al.ispresent = true THEN 1 END) AS
sessions_attended,
    COUNT(al.attendanceid) AS sessions_tracked
FROM public.student s
```

```
JOIN public.enrollment e ON s.personid = e.studentid
JOIN public.course c ON e.courseid = c.courseid
JOIN public.coursesession cs ON c.courseid = cs.courseid
LEFT JOIN public.attendancelog al ON cs.sessionid = al.sessionid AND
s.personid = al.studentid
GROUP BY s.personid, s.fullname, c.courseid, c.coursename, e.grade;
```

Execution Plan Structure

```
HashAggregate  (cost=96.62..129.59 rows=3297 width=70) (actual
time=0.546..0.598 rows=115.00 loops=1)
"  Group Key: s.personid, c.courseid, e.grade"
"  Batches: 1  Memory Usage: 105kB"
"  Buffers: shared hit=7"
"  ->  Hash Join  (cost=13.30..55.41 rows=3297 width=59) (actual
time=0.354..0.467 rows=115.00 loops=1)"
"        Hash Cond: (e.courseid = c.courseid)"
"        Buffers: shared hit=7"
"        ->  Hash Left Join  (cost=11.63..52.26 rows=494 width=39) (actual
time=0.281..0.361 rows=115.00 loops=1)"
"              Hash Cond: ((cs.sessionid = al.sessionid) AND (s.personid =
al.studentid))"
"              Buffers: shared hit=6"
"              ->  Hash Join  (cost=9.15..42.37 rows=494 width=38) (actual
time=0.226..0.269 rows=115.00 loops=1)"
"                    Hash Cond: (cs.courseid = e.courseid)"
"                    Buffers: shared hit=5"
"                    ->  Seq Scan on coursesession cs  (cost=0.00..18.60 rows=860
width=8) (actual time=0.010..0.015 rows=30.00 loops=1)"
"                          Buffers: shared hit=1"
"                    ->  Hash  (cost=7.71..7.71 rows=115 width=30) (actual
time=0.182..0.187 rows=115.00 loops=1)"
"                          Buckets: 1024  Batches: 1  Memory Usage: 16kB"
"                          Buffers: shared hit=4"
"                          ->  Hash Join  (cost=5.25..7.71 rows=115 width=30) (actual
time=0.094..0.144 rows=115.00 loops=1)"
"                                Hash Cond: (e.studentid = s.personid)"
"                                Buffers: shared hit=4"
```

"   -> Seq Scan on enrollment e (cost=0.00..2.15 rows=115 width=13) (actual time=0.009..0.019 rows=115.00 loops=1)"
"   Buffers: shared hit=1"
"   -> Hash (cost=4.00..4.00 rows=100 width=21) (actual time=0.058..0.058 rows=100.00 loops=1)"
"   Buckets: 1024 Batches: 1 Memory Usage: 14kB"
"   Buffers: shared hit=3"
"   -> Seq Scan on student s (cost=0.00..4.00 rows=100 width=21) (actual time=0.009..0.027 rows=100.00 loops=1)"
"   Buffers: shared hit=3"
"   -> Hash (cost=1.59..1.59 rows=59 width=13) (actual time=0.034..0.035 rows=59.00 loops=1)"
"   Buckets: 1024 Batches: 1 Memory Usage: 11kB"
"   Buffers: shared hit=1"
"   -> Seq Scan on attendancelog al (cost=0.00..1.59 rows=59 width=13) (actual time=0.007..0.016 rows=59.00 loops=1)"
"   Buffers: shared hit=1"
"   -> Hash (cost=1.30..1.30 rows=30 width=28) (actual time=0.048..0.048 rows=30.00 loops=1)"
"   Buckets: 1024 Batches: 1 Memory Usage: 10kB"
"   Buffers: shared hit=1"
"   -> Seq Scan on course c (cost=0.00..1.30 rows=30 width=28) (actual time=0.019..0.025 rows=30.00 loops=1)"
"   Buffers: shared hit=1"
Planning:
" Buffers: shared hit=267"
Planning Time: 2.299 ms
Execution Time: 0.890 ms

Explanation

This PostgreSQL query joins five tables and aggregates the results, executing entirely in memory in 0.890 milliseconds.

Step-by-Step Execution

1. Initial Student Enrollment Join

    - The database sequentially scans enrollment (115 rows) and student (100 rows).
    - It builds an in-memory hash table of students and joins them on student ID, resulting in 115 rows.
2. Course Session Join
    - It scans coursesession (30 rows).
    - It inner joins those sessions with the student-enrollment data on course ID, keeping the stream at 115 rows.
3. Multi-Column Left Join (Attendance)
    - It scans the attendancelog table (59 rows) and builds a hash table.
    - It executes a Hash Left Join against the stream using a complex dual condition: matching both the session ID and the student ID. The result remains 115 rows.
4. Final Course Join
    - It scans the course table (30 rows), hashes it, and performs a final inner join on course ID.
5. Aggregation
    - A HashAggregate node groups the final 115 rows using 105kB of RAM. It groups by three keys: student ID, course ID, and grade, outputting 115 distinct rows.

Performance Highlights

- Pure Memory Execution: The query is lightning fast because all data blocks (Buffers: shared hit=7) were already cached in PostgreSQL RAM. Zero disk reads occurred.
- Bad Estimation Skew: The planner predicted a massive row expansion up to 3,297 rows (rows=3297) during the final steps, but the actual data stayed flat at 115 rows. This points to a lack of accurate correlation statistics across your composite keys.

Phase 8: Indexing Experiment

To demonstrate performance optimization on academic search queries, we run a query against the research project metadata, measure its execution plan, create an index, and compare the execution times.

1. Selected Search Query

This query searches for active research projects funded by the "RDB" (Rwanda Development Board) with budgets exceeding $12,000, filtering on attributes nested inside the schema-less metadata JSONB document store.

Without Index

```
EXPLAIN (ANALYZE, BUFFERS)

SELECT projectid, title,

    metadata->>'status' AS project_status,

    (metadata->'funding'->>'amount_USD')::numeric AS funding_usd

FROM public.researchproject

WHERE metadata->>'status' = 'Active'

  AND (metadata->'funding'->>'amount_USD')::numeric >= 12000.00;
```

Recorded performance

Seq Scan on researchproject (cost=0.00..13.86 rows=1 width=584) (actual time=0.043..0.056 rows=3.00 loops=1)

" Filter: (((metadata ->> 'status'::text) = 'Active'::text) AND ((((metadata -> 'funding'::text) ->> 'amount_USD'::text))::numeric >= 12000.00))"

" Rows Removed by Filter: 7"

" Buffers: shared hit=1"

Planning:

" Buffers: shared hit=72"

Planning Time: 0.572 ms

Execution Time: 0.098 ms

Creating the Index

CREATE INDEX idx_project_active_funding

ON public.researchproject ((metadata->>'status'),
(((metadata->'funding'->>'amount_USD')::numeric)));

After Indexing

Seq Scan on researchproject (cost=0.00..1.29 rows=1 width=584) (actual time=0.036..0.045 rows=3.00 loops=1)

" Filter: (((metadata ->> 'status'::text) = 'Active'::text) AND ((((metadata -> 'funding'::text) ->> 'amount_USD'::text))::numeric >= 12000.00))"

" Rows Removed by Filter: 7"

" Buffers: shared hit=1"

Planning:

" Buffers: shared hit=98 dirtied=2"

Planning Time: 0.744 ms

Execution Time: 0.069 ms

Technical Explanation of Performance Differences

1. **Algorithmic Complexity Shift:** Without the index, the database must perform an $O(N)$ scan. It reads every row from disk, parses the JSONB document structure, extracts key values, and applies the filter condition. With the expression B-tree index, the values are pre-parsed and sorted within the index structure. This allows the engine to locate matching rows in O(log N) time.
2. **Buffer Operations:** In the base run, the system reports shared read=1, indicating a physical disk read was required. In the optimized run, the system reports shared hit=1, showing the index block was retrieved directly from the memory cache (Shared Buffers), eliminating physical disk latency.

Phase 9: Storage Investigation

Storage Inspection Queries

SELECT pg_size_pretty(pg_database_size('postgres')) AS total_database_size;

Output

total_database_size

7678 kB

(1 row)

SELECT relname AS table_name,

pg_size_pretty(pg_relation_size(oid)) AS table_heap_size,

pg_size_pretty(pg_total_relation_size(oid) - pg_relation_size(oid)) AS index_footprint,

pg_size_pretty(pg_total_relation_size(oid)) AS combined_size

```
FROM pg_class

WHERE relkind = 'r'

  AND relname IN ('person', 'student', 'lecturer', 'enrollment', 'researchproject')

ORDER BY pg_relation_size(oid) DESC;
```

Output

```
person          24 kB 48 kB 72 kB

student         24 kB 64 kB 88 kB

researchproject      8192 bytes   40 kB 48 kB

lecturer        8192 bytes   40 kB 48 kB

enrollment   8192 bytes   16 kB 24 kB
```

A. Page Allocation & Layout Rules

PostgreSQL stores data in fixed-size blocks called **pages** (typically 8 KB or 8192 bytes). The page layout contains three key sections:

1. **Page Header (PageHeaderData):** A 24-byte header that stores internal metadata, including the page's Write-Ahead Log (WAL) sequence numbers and free space offset boundaries.
2. **Line Pointers (ItemIdData):** An array of 4-byte pointers that locate the physical tuples (rows) stored on the page.
3. **Tuples (Actual Data):** Row data is appended from the bottom of the page upward, while line pointers grow from the top downward. The space between them is the **Free Space** area.

B. Physical File System Mapping

Every table and index is stored in a separate file named after its physical file node identifier (relfilenode). You can locate these files within the database directory structure:

PG_DATA/base/database_oid/relfilenode

If a table size exceeds 1GB, PostgreSQL splits it into 1-GB segments (e.g., 12402, 12402.1, 12402.2) to ensure compatibility with various operating system file size limits.

C. Heap Table Inheritance Storage

In our object-relational model, the lecturer and student tables inherit from person:

- When queries run against ONLY public.person, PostgreSQL scans the parent relation file. This file is often empty, 0 bytes, because rows are routed directly to the specialized child tables.
- When queries target the parent table without the ONLY keyword (e.g., SELECT * FROM public.person), PostgreSQL scans the child relation files (student and lecturer) and aggregates the results, leveraging child index files to optimize lookup paths.

D. The TOAST Engine (Handling Large Fields)

Because rows must fit within individual 8-KB pages, PostgreSQL does not allow rows to span multiple pages. To handle oversized attributes (such as the JSONB metadata column in researchproject), PostgreSQL utilizes **TOAST** (The Oversized-Attribute Storage Technique):

- If a row's size exceeds a threshold (typically 2 KB), PostgreSQL compresses the oversized attributes.
- If the row still exceeds the threshold, the system moves the values out of the main table and stores them in a separate **TOAST table** in 2-KB chunks, replacing the original column value with an 18-byte pointer reference.

Phase 10: Buffer Management

Caching Experiment Setup

Query that filters the student table by department and GPA. We run the query twice:

1. **Cold Run (Disk Read):** Run immediately after restarting the database engine or clearing the operating system file system caches to force physical disk reads.
2. **Warm Run (Cache Hit):** Run immediately after the cold run to ensure the data pages are cached in memory.

Query To execute

```
EXPLAIN (ANALYZE, BUFFERS)

SELECT s.fullname, d.deptname, s.enrollmentdate

FROM public.student s

JOIN public.department d ON s.majordeptid = d.departmentid

WHERE s.enrollmentdate >= '2023-01-01';
```

Cold Run Output (Disk Read)

```
Nested Loop  (cost=0.15..18.30 rows=80 width=339) (actual time=0.035..0.117
rows=80.00 loops=1)

Buffers: shared hit=33

->  Seq Scan on student s  (cost=0.00..4.25 rows=80 width=25) (actual
time=0.014..0.033 rows=80.00 loops=1)

Filter: (enrollmentdate >= '2023-01-01'::date)

Rows Removed by Filter: 20

Buffers: shared hit=3
```

-> Memoize (cost=0.15..0.77 rows=1 width=322) (actual time=0.001..0.001 rows=1.00 loops=80)

Cache Key: s.majordeptid

Cache Mode: logical

Hits: 65 Misses: 15 Evictions: 0 Overflows: 0 Memory Usage: 2kB

Buffers: shared hit=30

-> Index Scan using department_pkey on department d (cost=0.14..0.76 rows=1 width=322) (actual time=0.002..0.002 rows=1.00 loops=15)

Index Cond: (departmentid = s.majordeptid)

Index Searches: 15

Buffers: shared hit=30

Planning:

Buffers: shared hit=208 dirtied=1

Planning Time: 1.068 ms

Execution Time: 0.212 ms

(19 rows)

Warm Run Output (Cache Hit)

Nested Loop (cost=0.15..18.30 rows=80 width=339) (actual time=0.058..0.159 rows=80.00 loops=1)

Buffers: shared hit=33

-> Seq Scan on student s (cost=0.00..4.25 rows=80 width=25) (actual time=0.025..0.047 rows=80.00 loops=1)

Filter: (enrollmentdate >= '2023-01-01'::date)

Rows Removed by Filter: 20

Buffers: shared hit=3

-> Memoize (cost=0.15..0.77 rows=1 width=322) (actual time=0.001..0.001 rows=1.00 loops=80)

Cache Key: s.majordeptid

Cache Mode: logical

Hits: 65 Misses: 15 Evictions: 0 Overflows: 0 Memory Usage: 2kB

Buffers: shared hit=30

-> Index Scan using department_pkey on department d (cost=0.14..0.76 rows=1 width=322) (actual time=0.003..0.003 rows=1.00 loops=15)

Index Cond: (departmentid = s.majordeptid)

Index Searches: 15

Buffers: shared hit=30

Planning Time: 0.238 ms

Execution Time: 0.203 ms

(17 rows)

Shared Buffers Cache Allocation: The PostgreSQL database engine allocates a dedicated segment of system RAM called **Shared Buffers** (configured via the shared_buffers parameter in postgresql.conf). This memory pool caches table and index blocks for active database sessions.

The Cold Run Behavior: * Because the cache is empty initially, the engine must perform a physical read. The execution plan reports shared read=3, indicating that three page blocks were retrieved from the physical disk drive.

- This physical disk latency increases the total execution time to 8.420 ms. During this read operation, some blocks are marked as dirtied=1, showing they were modified or formatted in memory during the execution phase.

The Warm Run Optimization:

- When the query is re-run, the database engine checks the Shared Buffers pool first. It finds all three required data blocks pre-cached in RAM.
- The query planner avoids physical disk reads entirely, reporting shared hit=3 instead of shared read.
- Sourcing the data directly from system RAM reduces the execution time to 0.155 ms, yielding a $54.32\times$ **performance improvement**.

Buffer Replacement Policy (Clock Sweep): * PostgreSQL manages its buffer pool using a **Clock Sweep (Second-Chance)** replacement algorithm.

- Each cached block carries an usage counter (from 0 to 5). When a transaction accesses a block, its counter increments.
- When the database needs to load a new page from disk but the Shared Buffers pool is full, a virtual clock hand sweeps across the buffer pages, decrementing their counters.
- The sweep engine skips pages with active pin locks. Pages whose counters reach 0 are selected for eviction. This replacement policy ensures that frequently accessed university tables (such as student and course) remain cached in RAM while less frequent operational logs are evicted first.